

Stage 2 Report:

Predicting Helsinki City Bike Trip Duration from Route, Time and Weather

CS-C3240 Machine Learning

1. Introduction

Shared city bikes are popular in Helsinki and Espoo, but riders cannot easily tell in advance how long a trip will take. A good estimate would help people decide whether to cycle, for example in bad weather or at a busy time of day. This project predicts how long a city bike trip will take before it starts. Section 2 defines the problem and the data, Section 3 describes our methods, Section 4 presents the results, and Section 5 summarises what we learned.

2. Problem Formulation

This is a supervised regression problem. Each data point is one completed bike trip, and the label is the trip's duration in minutes. Each trip is described by seven features (Table 1): its route, when it starts, and the weather that day.

Table 1. Features describing each trip.

Feature	Type	Description
Route length	Continuous	Typical distance (m) between the start and end station, from past trips
Round trip	Binary	Whether the bike is returned to the station it was taken from
Weekday	Categorical	Day the trip started (Mon-Sun)
Hour of day	Categorical	Hour the trip started (0-23)
Month	Categorical	Month of the season (Apr-Oct)
Temperature	Continuous	Daily mean air temperature (°C)
Precipitation	Continuous	Daily total precipitation (mm)

The problem is the same as in Stage 1, but we added the route. In Stage 1 we left out distance entirely, so that the model could not see the answer. On reflection, that mixed up two different things. The distance a trip actually covered is only known after the ride, so we still leave it out. But riders know where they are going before they start, so the typical length of their route is fair to use.

The trip data comes from HSL's open dataset of city bike trips [1]. The weather data comes from the Finnish Meteorological Institute's open data for the Kaisaniemi station in Helsinki [2], and is matched to each trip by date.

3. Methods

3.1 Data and preprocessing

We use all trips from the April-October seasons of 2022-2025, about 10 million in total. We removed incomplete rows, "false starts" shorter than 10 seconds or 10 metres (a bike taken out and put straight back), and bikes moved to HSL's workshop or test stations. This leaves **9,837,352 trips**. We kept very long trips, most of which are bikes that were never properly returned, because there is no natural cut-off for removing them. Section 3.4 explains how we handle them instead.

3.2 Feature selection

We started from three assumptions: longer trips take longer, the time of day affects speed (through traffic or the kind of trip), and bad weather slows riders down. Since duration is distance divided by speed, we plotted the typical speed and the typical distance against each factor. Distance varied far more than speed, so trip length is the main thing to capture. Speed changed somewhat with the time of day: riders were fastest in the morning, when many of them are commuting, and slower in the afternoon. This suggests that the purpose of the trip matters more than traffic. Rain did not slow riders down, contrary to our Stage 1 assumption; on rainy days, people simply made shorter trips. We kept the Stage 1 time and weather features so that we could test their effect, and added two route features.

Route length is the median distance covered by other trips between the same two stations. To avoid leaking information, it is only computed from other seasons: 2022 training trips use 2023 trips and vice versa, validation trips use 2022-2023, and test trips use 2022-2024. Routes with fewer than three past trips use the median for their starting station instead. The **round-trip** flag marks trips that end where they started, which are mostly leisure rides.

3.3 Models

We compare two hypothesis spaces covered in the course [3]:

- **Linear regression.** We predict the logarithm of duration from the logarithm of route length and the other features. This matches the idea that duration is distance divided by speed, where time and weather make riders a few percent faster or slower.
- **Gradient-boosted regression trees** [5]. Trees can capture interactions that a linear model cannot, for example rain mattering more on long trips. We include them to check whether such interactions exist.

To see how much the models actually learn, we compare them with two simple baselines: always predicting the median duration, and a **physics baseline** that divides route length by one typical speed (11.5 km/h, estimated from the training data).

3.4 Loss functions

A small number of trips last hours or even days, mostly bikes that were never returned. With the usual squared error loss, these few trips would dominate training: trips longer than five hours are only 0.3% of the data, but they make up almost all of the squared error. We handle this in two ways. First, we predict the logarithm of the duration, which shrinks these extreme values. Second, we train the linear model with the **Huber loss** [4] ($\epsilon = 1.35$). It treats small errors like the squared error, but large errors only linearly, so a few extreme trips cannot pull the model too far. To check whether this matters, we also train the same linear model with the **squared error**. For the trees we use the **absolute error**, which is also robust to outliers; the library we used does not offer the Huber loss for trees.

This gives three methods: linear regression with squared error, linear regression with Huber loss, and trees with absolute error. We evaluate them in minutes, using the mean absolute error (MAE), the median absolute error (MedAE, the error on a typical trip) and the share of trips predicted within two minutes.

3.5 Training, validation and test sets

We split the data by season: 2022-2023 for training (4,876,755 trips), 2024 for validation (2,508,741 trips) and 2025 for testing (2,451,856 trips). This mimics real use, where the model predicts trips in a future season. A random split would put trips from the same days into both training and validation, which would make the results look better than they are. To keep training fast, every model is fitted on the same random sample of 400,000 training trips. We decided in advance to choose the method with the lowest validation MAE, retrain it on 2022-2024, and evaluate it once on the test set. Because the differences between methods are small, we also compute 95% confidence intervals for them by resampling whole days [6].

4. Results

4.1 Comparing the methods

Table 2. Errors in minutes, and share of trips predicted within 2 minutes. The chosen method is in bold.

Method	Training		Validation (2024)			Test (2025)		
	MAE	MedAE	MAE	MedAE	≤ 2 min	MAE	MedAE	≤ 2 min
Constant (median)	10.40	4.88	13.98	5.30	19.0%	14.89	5.43	18.6%
Physics baseline	7.92	1.76	11.22	2.06	49.1%	11.95	2.14	48.0%
Linear, squared error	7.74	1.98	11.00	2.21	46.1%	11.70	2.35	43.3%
Linear, Huber loss	7.62	1.62	10.91	1.86	52.5%	11.55	1.91	51.5%
Trees, absolute error	7.57	1.58	10.96	1.85	52.5%	11.60	1.89	51.8%

Table 2 shows four things:

- **Route length does most of the work.** The physics baseline only divides route length by a typical speed, yet it already cuts the typical validation error from 5.3 to 2.1 minutes.
- **Machine learning adds a small but real improvement.** The best methods bring the typical error down to about 1.85 minutes and predict 3.3 percentage points more trips within two minutes than the physics baseline (95% CI [3.1, 3.5]). Part of the gain comes from details the simple rule misses: longer routes are ridden faster, so doubling a route makes a trip about 80% longer rather than twice as long.
- **The Huber loss matters.** With squared error, the few extreme trips still pull predictions up by about half a minute, so this method does worse than the physics baseline on typical trips. With the Huber loss, 6.4 percentage points more trips are predicted within two minutes (CI [6.1, 6.6]). Removing the longest 0.1% of training trips also moves the squared-error model's predictions about ten times more than the Huber model's.
- **Trees do not help.** The trees and the Huber model are practically tied. The linear model's MAE is 0.05 minutes (about three seconds) lower, and both predict the same share of trips within two minutes.

Training errors are lower than validation errors for every method, including the constant baseline, which learns almost nothing. The gap therefore comes from 2024 being a harder season, not from overfitting. We choose **linear regression with the Huber loss**: it has the lowest validation MAE, as our rule requires, and it is simpler and easier to interpret than the trees.

4.2 Which features matter

To test our three assumptions, we refitted the chosen model with different sets of features (Table 3).

Table 3. Validation results of the chosen model with different features. "Route" means route length and the round-trip flag.

Features used by the chosen model	Validation MedAE (min)	Within 2 min
None (constant prediction)	5.300	19.0%
Time + weather (the Stage 1 features)	5.250	19.1%
Route	1.865	52.3%
Route + time	1.850	52.6%
Route + weather	1.875	52.2%
Route + time + weather (chosen model)	1.857	52.5%

Route length explains almost everything. Without it, the Stage 1 features barely beat a constant prediction; with route length alone, the model is almost as good as the full model. Time adds a little: on the same route, a trip takes about 10% less time at 5-6 am and about 4% more in the early afternoon than on average, and weekend trips take about 3% longer. Weather adds nothing, and adding it to route length even makes the typical error slightly worse. In the fitted model, 10 mm of rain makes a trip only about 1% shorter.

4.3 Test error

The 2025 test season was not used for training or for choosing the method. On it, the chosen method has a **test MAE of 11.55 minutes and a test MedAE of 1.91 minutes**, and predicts 51.5% of trips within two minutes (Table 2). This is close to its validation results, so the model also works on a new season, and the methods rank the same way as on the validation data. The predictions are not biased: on a typical trip, the ratio of actual to predicted duration is 1.006.

5. Conclusions

We predicted city bike trip durations from the route, the time and the weather. The chosen method, linear regression with the Huber loss, has a typical error of 1.9 minutes on an unseen season, against 5.4 minutes for a constant guess. The main lesson is that route length explains most of the prediction: a simple distance-divided-by-speed rule already does well, and machine learning adds a small but real improvement on top. Training with the Huber loss matters because of the extreme trips, while more complex trees and the weather add little.

For a typical trip, the problem is solved well. The average error (about 11.5 minutes) stays much larger than the typical error because it is dominated by bikes that were never returned and by riders who stop along the way, which our features cannot predict. The results could be improved with hourly instead of daily weather, station coordinates to estimate the length of routes without history, training on all trips instead of a sample, and a separate model that detects bikes that were not returned.

Use of AI

We used generative AI (Claude, by Anthropic) throughout Stage 2. It helped us plan and carry out the analyses, write and debug the notebook code, think through the problem formulation (including the difference between a trip's recorded distance and its route), and draft and edit this report. We made the main design decisions ourselves: adding route length, comparing three methods, and computing route length from past trips. We checked the code, results and text before submitting.

6. References

- [1] HSL (Helsinki Region Transport). Helsinki and Espoo city bike trip data (CC BY 4.0). https://hri.fi/data/en_GB/dataset/helsingin-ja-espoon-kaupunkipyorilla-ajatut-matkat
- [2] Finnish Meteorological Institute. Open data, weather observations (CC BY 4.0). <https://en.ilmatieteenlaitos.fi/open-data>
- [3] Jung, A. Machine Learning: The Basics. <https://mlbook.cs.aalto.fi>
- [4] Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1), 73-101.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [6] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

7. Appendix: Code

The full notebook (ML_bike.ipynb), which prepares the data and trains and evaluates all methods, is attached on the following pages.